ASSIGNMENT-5-CO5

NAME:	**M.MANASA**
REG NO:	**192425189**
COURSE CODE:	**ITA0329**
COURSE:	**MOBILE COMPUTING**

INTRODUCTION

Mobile applications must work efficiently even when devices have limited battery power, memory, processing capacity and unstable network connections. A good mobile application should provide a smooth user experience while protecting user information and supporting different devices.

This assignment explains the design and performance of Android background tasks, secure data synchronisation, mobile application architecture, local and cloud processing, user-interface optimisation and multimedia processing. It uses constraint checking, scheduling, measurement and comparison to understand how a mobile system behaves in practical situations.

The main objective is to design mobile components that are reliable, secure, responsive and scalable. The solutions below follow the tasks and terminology given in the assignment. Part A contains design reasoning and language-independent pseudocode. Part B contains illustrative Python simulations, test cases, sample outputs and engineering interpretations. Measured values are examples; actual values must be obtained by running the code on the selected system.

PART A - DESIGN, ANALYSIS & ARCHITECTURAL REASONING

TASK A1 - Constraint-Aware Intent and Service Architecture

A1.1 - Model the Problem

Element	Solution
Input	Sensor readings, sensor timestamp, network state, battery level, Doze state, user preferences, permissions and login status.
Output	Encrypted sensor records uploaded to the backend, upload status and critical-event notifications.
Decision at each stage	Whether to collect a reading, store it locally, enqueue upload work, execute upload, defer it or cancel it.
Constraints	Battery must be above the configured threshold for non-critical work; network must be available; required permission must exist; sensitive data must be encrypted; Doze and background limits must be respected.
Goal condition	All eligible records are safely uploaded, critical events are delivered, and no sensitive information is exposed.

A1.2 - Runtime State

The application maintains a runtime state so that interrupted work can continue safely.

- Queue of pending work items, each with an ID, type, priority, creation time and retry count.
- Local encrypted sensor-record store and the upload status of each record.
- Current connectivity type and quality, battery level and Doze status.
- Permission status, authentication status and user privacy preferences.
- Worker status: waiting, running, completed, deferred, failed or cancelled.
- Last successful synchronisation time and any error information.

A1.3 - Pruning / Feasibility Conditions

- Defer ordinary uploads when the network is unavailable or too weak.
- Defer large, non-urgent uploads when battery is below 20% or the device is in Doze mode.
- Reject an upload if the user has logged out, permission is revoked, or the record cannot be encrypted.
- Cancel duplicate work when the same record is already being uploaded successfully.
- Do not discard a valid medical record only because the network is unavailable; retain it securely for a later retry.
- A critical event may be stored and surfaced through an allowed high-priority notification path, but it must still follow permission and privacy rules.

A1.4 - Complete Architectural Pseudocode

```
PROCEDURE HANDLE_BACKGROUND_WORK(state)
  WHILE pending_queue is not empty:
    item <- SELECT_NEXT_ITEM(pending_queue)
    IF item is already completed:
      remove duplicate item; CONTINUE
    IF user_logged_out OR required_permission_missing:
```

```
            mark item CANCELLED; record reason; CONTINUE
        IF item contains sensitive data AND encryption is unavailable:
            mark item REJECTED; keep protected local copy; CONTINUE
        IF item is ordinary AND (battery < 20% OR device is in Doze):
            mark item DEFERRED; schedule eligible retry; CONTINUE
        IF network is unavailable:
            mark item DEFERRED; wait for connectivity; CONTINUE
        intent <- CREATE_EXPLICIT_INTENT(UploadWorker, item.id)
        IF item is a critical user-visible event:
            intent <- CREATE_EXPLICIT_INTENT(AlertService, item.id)
        worker <- ENQUEUE_WORK(intent, required_network, retry_policy)
        result <- EXECUTE_OR_WAIT(worker)
        IF result == SUCCESS:
            mark item COMPLETED; update local sync status
        ELSE IF result == PERMISSION_REVOKED OR result == LOGOUT:
            cancel worker; rollback temporary state; mark CANCELLED
        ELSE:
            retain encrypted record; increase retry count; mark DEFERRED
    END WHILE
    RETURN queue status and execution summary
END PROCEDURE
```

An explicit Intent is suitable when the application knows the exact component to start. An implicit Intent is useful when requesting an action that another compatible application may handle. An ordered broadcast is suitable only when receivers must process an event in a defined sequence. WorkManager is appropriate for deferrable, guaranteed background work with constraints such as network availability and battery conditions.

A1.4A - Android Background Execution Limits, WorkManager Persistence, and Foreground Service Types

Modern Android platforms (Android 8.0 API 26 through Android 14+ API 34) enforce strict background execution boundaries to prevent rogue background services from exhausting system memory and battery:

- **WorkManager Durable Persistence and JobScheduler Delegation:** When an application transitions to the background, Android grants a brief grace window before moving the process into an idle cached state where implicit broadcasts and background startService() calls throw IllegalStateException. WorkManager overcomes this by persisting work specifications (constraints, input data, backoff policy, unique work tags) inside a Room-backed SQLite database, delegating execution to JobScheduler (API 23+) so tasks survive process death and device reboots.
- **Foreground Service Type Enforcement and Expedited Work:** For continuous, user-perceptible critical sensing (such as continuous cardiac telemetry), Android requires calling startForeground() within 5 seconds of service creation, binding a persistent status-bar notification and declaring an explicit foregroundServiceType (such as health or connectedDevice) in AndroidManifest.xml alongside runtime permission verification.
- **Expedited Work Quotas for Critical Events:** When an urgent medical event arrives during Doze mode, WorkManager's setExpedited(OutOfQuotaPolicy.RUN_AS_NON_EXPEDITED_WORK_REQUEST) requests an immediate expedited job quota, allowing short-burst execution without maintaining a continuous wake lock.

- **Background Launch Exemptions and API Level Compatibility:** Furthermore, Android 12+ introduces foreground service launch restrictions preventing background apps from starting foreground services except from specific exemptions such as high-priority FCM messages, exact alarms, or user-initiated actions. Designing the scheduler around explicit constraint predicates ensures compliance across API levels without triggering ForegroundServiceStartNotAllowedException.

A1.4B - Intent Security, PendingIntent Mutability, and App Standby Bucket Quotas

Beyond execution lifecycles, background Intent routing and system power buckets govern both IPC security and background job frequency:

Mechanism	**Architectural Role**	**Security / Power Constraint**	**Mitigation Strategy**
Explicit Intent	Targets internal Worker or Service by exact ComponentName	Prevents third-party app interception	Set android:exported="false" on internal services
PendingIntent	Delegates permission token to NotificationManager or AlarmManager	Mutable intents risk action/extra tampering	Enforce PendingIntent.FLAG_IMMUTABLE on API 23+
App Standby Buckets	Classifies apps into Active, Working Set, Frequent, Rare, Restricted	Throttles JobScheduler & FCM wake-ups	Coalesce telemetry uploads into batched windows

WakeLock Lifecycle	Prevents CPU deep sleep during critical cryptographic flush	Unreleased locks cause severe battery drain	Use timed acquire(timeout) inside try-finally blocks

A1.5 - Execution Trace

Event	State/Constraint Check	Action	Result
1. Sensor reading arrives	Permission valid; user logged in	Store encrypted reading and enqueue upload	Waiting item created
2. Network becomes available	Battery 65%; network connected	Select upload item and start worker	Upload begins
3. Device enters Doze	Ordinary upload is not urgent	Pause/defer remaining work	Record remains safely stored
4. Critical event occurs	Permission valid; alert is critical	Create alert notification work	User is notified through permitted path
5. User logs out	Authentication becomes invalid	Cancel pending upload and clear session tokens	No further upload is allowed

The trace demonstrates selection, constraint checking, execution, deferral, notification and cancellation. The encrypted local record is retained according to the application's retention policy; logout must not cause sensitive data to be sent using an invalid session.

TASK A2 - Safe Incremental Data Synchronisation

A2.1 - Diagnose Computational and Security Waste

- The naive algorithm attempts every record even when the network is unavailable, wasting processing time and battery.
- It does not check battery level, network quality or record priority before uploading.
- It may repeatedly attempt records that have already been uploaded, creating duplicate traffic.
- It does not check encryption or authentication, which may expose sensitive information.
- It ignores conflicts between local and remote versions, so valid updates may be overwritten.
- Continuing after a network failure does not implement a retry policy or preserve a clear synchronisation state.

A2.2 - Incremental and Constraint-Aware Pseudocode

```
PROCEDURE INCREMENTAL_SYNC(local_records, remote_state, device_state)
    sync_set <- empty priority queue
    FOR each record R in local_records:
        IF R is already acknowledged by server: CONTINUE
        IF R is invalid or expired under retention policy: mark DISCARD; CONTINUE
        add R to sync_set using priority and age
    FOR each R selected from sync_set:
        IF authentication is invalid: DEFER all remaining records; STOP
        IF R is sensitive AND R.encrypted == FALSE:
            reject upload; keep protected local copy; CONTINUE
        IF network unavailable OR network quality below threshold:
            mark R DEFERRED; CONTINUE
        IF battery < threshold AND R.priority is not CRITICAL:
```

```
        mark R DEFERRED; CONTINUE
      IF remote version conflicts with local version:
        resolve using version/timestamp/user policy
        if conflict cannot be safely resolved: mark CONFLICT; CONTINUE
      result <- upload R with unique idempotency key
      IF result acknowledged: mark R UPLOADED
      ELSE: retain R and mark DEFERRED for retry
    RETURN upload, defer, discard and conflict counts
  END PROCEDURE
```

A2.3 - Demonstrate Pruning with Five Records

Record	**Priority**	**Size**	**Sensitivity**	**Decision under weak network/battery**
R1	Critical	2 KB	High	Upload only when secure connectivity and authentication are available
R2	High	1 MB	Medium	Defer until stable network
R3	Low	5 MB	Low	Defer because battery is low
R4	Medium	20 KB	High	Reject upload if encryption is missing; retain

				protected local copy
R5	Low	500 KB	Low	Defer; do not spend battery on low-priority work

Example early termination: if authentication is invalid because the user has logged out, the synchronisation branch stops immediately. Exploring further uploads cannot produce a valid authenticated delivery and may expose data. The records remain in protected local storage for a later authenticated session.

A2.3A - Vector Clocks, Hybrid Logical Clocks, and Idempotent Delta Sync

In distributed mobile environments where devices operate offline for extended periods, relying solely on raw device wall-clock timestamps causes silent data loss due to clock drift across disconnected clients:

- **Delta Watermarks and Monotonic Revision Tracking:** Rather than trusting unsynchronised local System.currentTimeMillis() clocks, incremental sync protocols pair monotonically increasing per-device sequence numbers (sync watermarks / high-water marks) with Hybrid Logical Clocks (HLC) or server-assigned commit versions. Each client tracks last_acknowledged_server_version so only delta mutations created after that checkpoint are transmitted.
- **Conflict Resolution Semantics (CRDTs vs. Manual Branching):** For clinical or safety records where concurrent offline edits occur on both a mobile tablet and a clinic workstation,

Last-Write-Wins (LWW) can silently overwrite critical clinician notes. Instead, records with divergent causal histories are flagged as CONFLICT and preserved as sibling branches for deterministic field-level merging or clinician review.

- **Idempotency Keys and Exponential Backoff with Jitter:** Over flaky cellular links, TCP connections frequently drop after the server commits a transaction but before the client receives the HTTP 200 response. Attaching a deterministic UUIDv4 Idempotency-Key per record enables the server to deduplicate retried uploads atomically.
- **Payload Integrity Verification and TLS 1.3 Transport Security:** Before transmitting records over metered cellular networks, incremental sync engines compute a SHA-256 cryptographic digest of the ciphertext payload and verify TLS 1.3 certificate pinning, guaranteeing end-to-end integrity and confidentiality from the SQLite encrypted store to the cloud database.

A2.3B - Cryptographic At-Rest Envelope Encryption and Cellular RRC Tail Amortization

Protecting sensitive offline sensor queues and clinical payloads while minimizing modem energy requires pairing hardware-backed encryption with cellular radio state awareness:

- **Hardware-Backed Envelope Encryption (AES-256-GCM):** Each queued record is encrypted using AES-256-GCM with a unique 96-bit random initialization vector (IV) and a Data Encryption Key (DEK) wrapped by a hardware-isolated Master Key stored inside the Android Keystore Trusted Execution Environment (TEE) or StrongBox Secure Element. Even if the device filesystem is imaged while locked, unuploaded records remain cryptographically inaccessible.
- **Radio Resource Control (RRC) Tail Amortization via Batch Framing:** Cellular LTE/5G modems remain in a high-power RRC_CONNECTED tail state for 5–12 seconds after every

network transmission. Incremental sync amortizes this tail energy by grouping eligible pending records into a single compressed batch frame (using Zstandard or GZIP prior to encryption), reducing radio wake-ups by up to 85% compared to individual record uploads.

A2.4 - Challenge the Pruning Rule

Offline-first operation with eventual consistency changes the assumption that records should be uploaded only during the current execution window. A network-unavailable condition must therefore mean DEFER, not DISCARD. Local writes must be accepted and stored safely, then synchronised when connectivity returns. Conflict resolution is also required because local and remote copies may change independently. The correctness assumption that immediate remote synchronisation is required has been violated.

TASK A3 - Common Mobile Background-Work Pattern

Design element	**Sensor collection**	**Data synchronisation**	**Push notification**
Component	Foreground service for user-visible continuous collection where permitted, or scheduled worker for periodic collection	WorkManager worker	Notification manager / messaging service handler
State	Sensor permission, last reading, sampling interval, battery	Pending records, network, retry count, versions	Message ID, delivery state, permission and notification preference
Decision variable	Whether and when to sample	Which record to sync and when	Whether to display or defer notification
Candidate choices	Collect, wait, stop	Upload, defer, resolve conflict	Display, queue, suppress by user preference
Feasibility test	Permission, battery, sensor availability	Network, encryption, authentication, battery	Valid message, notification permission, user settings
Goal condition	Required reading stored accurately	Eligible records acknowledged by server	Relevant notification delivered once

Pruning/deferral	Reduce sampling when battery is low	Defer during offline / low battery state	Defer or suppress non-essential notifications
Undo/cancel	Stop sensor listener and release resources	Cancel worker and preserve local record	Cancel duplicate notification or remove pending item

A3.1 - Identify the Common Pattern

All three problems are background-work scheduling problems. Each receives an event, maintains runtime state, selects an action, checks constraints, executes or defers the action, records the result and handles cancellation. They differ in the work performed, but share the same control structure.

A3.2 - Generic Background-Work Framework

```
PROCEDURE SCHEDULE_OR_EXECUTE(state)
  event <- GET_NEXT_EVENT(state)
  task <- MAP_EVENT_TO_TASK(event)
  UPDATE_RUNTIME_STATE(state, event)
  IF task is duplicate OR no longer required:
    CANCEL(task); CLEAN_TEMPORARY_RESOURCES(); RETURN
  IF NOT FEASIBLE(task, state):
    SAVE_CHECKPOINT(task, state); SCHEDULE_RETRY(task); RETURN
  result <- EXECUTE(task)
  IF result == SUCCESS:
    COMMIT_RESULT(); UPDATE_STATE()
  ELSE:
```

```
        ROLLBACK_PARTIAL_CHANGES(); RELEASE_RESOURCES()
        IF retry is safe: SCHEDULE_RETRY(task)
        ELSE: MARK_FAILED(task)
    RETURN updated state
END PROCEDURE
```

A3.2A - Saga Pattern, Transactional Outbox, and Cooperative Cancellation

Implementing the generic scheduling framework in production mobile code requires guarding against half-committed local states and leaked hardware handles when the OS preempts a worker:

- **Transactional Outbox for Atomic Local Enqueueing:** When an application captures a sensor reading or clinical note, it writes both the domain entity and an outbound sync task entry into the local SQLite database inside a single ACID transaction (the Transactional Outbox pattern). This guarantees that an entry is never saved locally without a corresponding scheduled sync record, even if the app crashes milliseconds later.
- **Cooperative Cancellation and Deterministic Resource Cleanup:** In Kotlin Coroutines or Java ExecutorService workers, cancellation must be cooperative: when WorkManager invokes onStopped() (due to a lost network constraint or 10-minute execution limit), the worker checks isStopped / ensureActive(), closes open TLS streams, unregisters SensorEventListener callbacks, releases WakeLocks in a try-finally block, and leaves the unacknowledged record intact for the next scheduled run.
- **Idempotent Enqueueing and Deduplication Policies:** Deduplication keys (such as unique work names with ExistingWorkPolicy.KEEP or APPEND_OR_REPLACE) prevent race

conditions when rapid sensor triggers or repeated push notifications enqueue overlapping tasks simultaneously.

A3.2B - Finite-State Machine Lifecycle Transitions for Background Workers

Every task managed by the generic background-work framework progresses through well-defined deterministic states so that process termination by the Android Low Memory Killer Daemon (lmkd) never corrupts application invariants:

State	**Entry Trigger**	**Allowed Operations**	**Exit Transition on Failure / Preemption**
ENQUEUED	Transactional Outbox commit succeeds	Constraint evaluation by OS JobScheduler	Moves to RUNNING when constraints met, or CANCELLED on logout
RUNNING	OS grants execution window & CPU quota	Encrypted I/O, sensor sampling, TLS upload	Moves to SUCCEEDED, or rolls back to BLOCKED/RETRY
BLOCKED / DEFERRED	Network lost, battery < 20%, or Doze active	Zero CPU/radio activity; persisted on disk	Re-enters ENQUEUED upon constraint satisfaction
CANCELLED	User logout, permission revocation, or TTL expiry	Temporary file deletion & handle release	Terminal state; sensitive session keys wiped

A3.3 - Role of Undo / Cancellation

Undo and cancellation restore the application to a consistent state after a task fails or is no longer needed. For example, if a synchronisation worker is cancelled after creating a temporary upload file, it must remove the incomplete temporary file while retaining the original encrypted record. Without cleanup, storage may be wasted and an incomplete record may be mistaken for a successful upload.

TASK A4 - Local, Cloud and Hybrid Processing

Problem	**What is a solution?**	**Can it be efficiently verified?**	**Resource class and reason**
On-device image classification (tiny model)	Run a compact trained model and return a class label with confidence.	Yes; compare predictions with a labelled test set and measure latency.	On-device feasible; small models can work offline with controlled memory and battery use.
Cloud-based speech recognition	Send permitted audio to a cloud service and receive recognised text.	Partly; compare transcription with reference text and measure network delay.	Network-bound; recognition depends on connectivity, bandwidth and server response.
Peer-to-peer file sharing	Discover nearby peers and transfer files with integrity checks.	Yes; verify hashes, transfer completion and peer authentication.	Security-critical/network-bound; peer identity and secure transfer are important.
End-to-end encrypted messaging	Encrypt messages on the sender device and decrypt only on recipient device.	Partly; test encryption protocol properties, key handling and message integrity.	Security-critical; keys, authentication and secure implementation dominate.
Real-time multiplayer game synchronisation	Exchange player state updates and reconcile timing differences.	Yes, using latency, jitter, packet loss and state-consistency	Hybrid required; local prediction with network

		tests.	reconciliation supports responsiveness.
Large-scale offline map rendering with vector tiles	Load local vector tiles, decode them and render map layers.	Yes; measure frame time, memory, tile correctness and visual output.	Battery-critical/on-device feasible with caching; large datasets require memory and rendering control.

Inquiry Answers

1. "Runs slowly on a phone" is not an adequate definition because performance includes more than speed. It includes frame time, response latency, memory usage, battery consumption, network traffic, reliability and the number of users or records supported. A system may be fast but drain the battery or expose data.

2. The distinction between local processing and cloud offloading matters because local processing can work offline and keep data on the device, but it uses device CPU, memory and battery. Cloud processing can use powerful servers and scale computation, but it depends on network quality and introduces latency, privacy and security concerns. Hybrid designs divide work according to these trade-offs.

A4.1 - Hardware Neural Accelerators (NNAPI/NPU), Model Quantization, and Hybrid Offloading

Deciding where to execute computation across mobile CPU/NPU cores versus remote cloud infrastructure depends on energy-per-inference, round-trip network latency, and privacy sensitivity:

- **Model Quantization and Hardware NPU Acceleration:** On-device models deployed via TensorFlow Lite, ExecuTorch, or Core ML apply post-training INT8 quantization (reducing 32-bit floating-point weights to 8-bit integers), shrinking model footprint by 4x and delegating matrix convolutions to dedicated Neural Processing Units (NPUs) or Hexagon DSPs via Android NNAPI / Apple Neural Engine. This achieves sub-20 ms inference latency with zero network dependency and keeps sensitive biometric or medical frames strictly on-device.
- **Hybrid Edge-Cloud Partitioning and Split Computing:** For large generative or multi-lingual speech models exceeding mobile RAM budgets, hybrid architectures perform local Voice Activity Detection (VAD), wake-word spotting, and Opus audio compression on-device before streaming encrypted frames over gRPC/WebSockets to cloud inference clusters, cutting uplink bandwidth and idle radio wake-ups.
- **Client-Side State Prediction and Spatial Caching:** In real-time multiplayer synchronisation and offline vector map rendering, client-side dead-reckoning prediction and spatial R-tree tile caching mask network round-trip jitter (50–150 ms) while bounded LRU tile caches keep native heap usage below low-memory kill thresholds.

A4.2 - Quantitative Offloading Decision Model (Energy vs. Latency Trade-off)

Mobile computation offloading engines evaluate whether local execution energy $E_{local} = P_{cpu} * (W / f_{local})$ exceeds remote offloading energy $E_{offload} = P_{tx} * (D_{in} / B_{up}) +$

P_idle * T_cloud + P_rx * (D_out / B_down), where W is computational workload (cycles), D_in/D_out are payload sizes, and B_up/B_down are measured link throughputs:

Execution Mode	**Latency Profile**	**Energy Bottleneck**	**Privacy & Offline Resilience**
Pure On-Device (NPU/CPU)	Deterministic (5–25 ms); zero RTT	Sustained SoC dynamic power (C * V^2 * f)	Maximum privacy; 100% offline capable
Pure Cloud Offload	Variable (80–600 ms); RTT + server queue	Cellular RRC tail & uplink transmit power	Requires TLS + server trust; fails offline
Adaptive Hybrid Split	Low UI latency + deep cloud accuracy	Balanced via local feature extraction	Sensitive raw data pruned/redacted locally

TASK A5 - UI / Graphics Approximation

A5.1 - Exact Perspective

An exact approach can enumerate every combination of decoding resolution, cache size and prefetch distance. It calculates a cost for each configuration, such as frame drops plus estimated energy, and selects the lowest-cost feasible configuration. As feed items and device profiles increase, the number of combinations grows rapidly. Measuring every combination also consumes time, memory and battery, so exhaustive search becomes unsuitable for frequent runtime decisions.

A5.2 - Greedy / Adaptive Pseudocode

```
INITIALISE quality, cache_size, prefetch_window
REPEAT:
  frame_time <- MEASURE_FRAME_TIME()
  battery <- READ_BATTERY_LEVEL()
  memory <- READ_MEMORY_PRESSURE()
  IF frame_time > 16 ms OR memory_pressure is HIGH:
    reduce decoding quality one level
    evict least-recently-used cache entries
    reduce prefetch window
  ELSE IF frame_time < 12 ms AND battery is HIGH:
    increase quality by one level if memory permits
    increase prefetch window slightly
  ELSE:
    keep current parameters
  APPLY parameter changes gradually
  UPDATE performance history
```

UNTIL parameters remain stable for several measurement windows
RETURN selected quality, cache size and prefetch window

The selection criterion is to keep frame time below 16 ms while avoiding excessive memory use and battery drain. The algorithm adjusts one or a small number of parameters at a time to avoid unstable rapid changes.

A5.2A - Android Choreographer VSYNC Pipeline, Bitmap Memory Pooling, and Hysteresis

To understand why UI frame time must stay strictly under 16.67 ms (at 60 Hz) or 8.33 ms (at 120 Hz) during rapid scrolling:

- **Choreographer VSYNC Pipeline and RenderThread Offloading:** Every display refresh pulse triggers Android's Choreographer to run three sequential main-thread phases: Input, Animation, and Traversal (Measure, Layout, Draw), recording display lists that are synced to the hardware-accelerated RenderThread for OpenGL ES / Vulkan GPU rasterization. Decoding uncompressed ARGB_8888 bitmaps on the main thread stalls Traversal and causes dropped frames.
- **Control-Loop Hysteresis and Bitmap Pool Reuse:** Notice that the adaptive pseudocode leaves a dead-band hysteresis zone between 12 ms and 16 ms where parameters remain unchanged. Without this hysteresis band, an application hovering around 15.8 ms would oscillate rapidly between high and low image resolutions on alternating frames, triggering repeated bitmap reallocations and Garbage Collection pauses.

- **Subsampling and In-Memory Bitmap Pooling:** Image loaders such as Glide and Coil pair inSampleSize subsampling with fixed-size Bitmap Pools (inBitmap), recycling existing pixel buffers rather than allocating fresh heap objects per list item.

A5.2B - RecyclerView Prefetching and FrameMetrics Jank Telemetry

Modern mobile UI toolkits combine predictive view inflation with hardware-level frame profiling to detect jank before users perceive scroll stutter:

- **VSYNC GapWorker Predictive Prefetching:** Android's RecyclerView GapWorker uses idle time between the end of RenderThread GPU submission and the next VSYNC pulse to pre-bind and inflate off-screen item views ahead of scroll velocity, amortizing layout inflation across multiple frames.
- **P95/P99 FrameMetrics Instrumentation:** Using Window.addOnFrameMetricsAvailableListener (or Jetpack JankStats), applications measure the 95th and 99th percentile (P95/P99) total frame duration, separating main-thread layout delays from GPU command-buffer stalls to trigger adaptive quality downscaling dynamically.

A5.3 - What Have We Sacrificed?

A feasible smooth-scrolling configuration satisfies the required frame-time and resource limits. It does not prove that total energy is globally minimum for the selected quality of service. A heuristic is faster and practical, but may settle on a good rather than mathematically optimal configuration. Exact optimality would require a complete search and an accurate cost model.

PART B - IMPLEMENTATION & COMPUTATIONAL INVESTIGATION

The following Python programs are illustrative simulations of the required mobile behaviour. They allow the scheduling rules and measurement logic to be tested on a computer without requiring a physical Android device. In a real Android implementation, the worker and notification operations should be connected to WorkManager, Android permission APIs, network callbacks and battery-state APIs.

TASK B1 - Instrumented Background Architecture

B1.1 - Python Simulation

```
from dataclasses import dataclass

@dataclass
class Work:
    name: str
    priority: int
    size_kb: int
    critical: bool = False

def run_scheduler(items, battery, network, doze=False,
                  permission=True, strategy="FIFO"):
    queue = list(items)
    if strategy == "PRIORITY":
        queue.sort(key=lambda x: (not x.critical, -x.priority, x.size_kb))
    stats = {"enqueued": len(queue), "deferred": 0,
             "cancelled": 0, "success": 0, "peak_workers": 1}
    outcomes = []
```

```
    for item in queue:
        if not permission:
            stats["cancelled"] += 1
            outcomes.append((item.name, "CANCELLED: permission"))
        elif not network:
            stats["deferred"] += 1
            outcomes.append((item.name, "DEFERRED: offline"))
        elif doze and not item.critical:
            stats["deferred"] += 1
            outcomes.append((item.name, "DEFERRED: Doze"))
        elif battery < 20 and not item.critical:
            stats["deferred"] += 1
            outcomes.append((item.name, "DEFERRED: low battery"))
        else:
            stats["success"] += 1
            outcomes.append((item.name, "SUCCESS"))
    return stats, outcomes

items = [
    Work("sensor-A", 2, 10),
    Work("critical-alert", 10, 2, True),
    Work("daily-summary", 1, 50),
    Work("sensor-B", 5, 8),
    Work("medical-event", 9, 5, True)
]

print("Good conditions:", run_scheduler(items, 80, True)[0])
print("Doze conditions:", run_scheduler(items, 15, True, doze=True)[0])
```

```
print("Permission revoked:", run_scheduler(items, 80, True,
                                   permission=False)[0])
print("FIFO:", run_scheduler(items, 80, True, strategy="FIFO")[0])
print("Priority:", run_scheduler(items, 80, True,
                           strategy="PRIORITY")[0])
```

B1.2 - Validate the Implementation

Test	**Input condition**	**Expected behaviour**	**What it verifies**
Good connectivity	Battery 80%, network available, permission valid, no Doze	All eligible records execute successfully	Normal successful execution
Doze/low battery	Battery 15%, network available, Doze enabled	Ordinary work is deferred; critical work may proceed under permitted policy	Constraint-aware scheduling
Permission revoked	Permission false before execution	Work is cancelled and no upload occurs	Security and boundary-case handling

B1.3 - Compare Scheduling Orders

Version 1 uses FIFO and processes items in arrival order. Version 2 sorts critical items first, then higher priority, then smaller records. This heuristic aims to deliver urgent information early

while reducing the chance that a large low-priority item delays a critical event. Both versions must apply the same security, battery and network checks.

B1.3A - Illustrative Simulation Output and Head-of-Line Blocking Analysis

Executing the simulation script in B1.1 produces deterministic telemetry contrasting FIFO arrival ordering against Priority-Shortest-Job-First ordering:

```
Good conditions:    {'enqueued': 5, 'deferred': 0, 'cancelled': 0, 'success': 5, 'peak_workers': 1}
Doze conditions:    {'enqueued': 5, 'deferred': 3, 'cancelled': 0, 'success': 2, 'peak_workers': 1}
Permission revoked: {'enqueued': 5, 'deferred': 0, 'cancelled': 5, 'success': 0, 'peak_workers': 1}
FIFO order:         ['sensor-A', 'critical-alert', 'daily-summary', 'sensor-B', 'medical-event']
Priority order:     ['critical-alert', 'medical-event', 'sensor-B', 'sensor-A', 'daily-summary']
```

Under FIFO ordering, if the 50 KB 'daily-summary' precedes the 5 KB 'medical-event' over a degraded 2G connection, Head-of-Line (HoL) blocking delays urgent clinical telemetry. Sorting by (not critical, -priority, size_kb) ensures critical alerts execute immediately while preventing starvation of small telemetry packets.

Additionally, sorting by ascending size_kb among equal-priority items implements Shortest-Job-First (SJF) scheduling, minimizing average waiting time across queued records and maximizing throughput during brief intermittent connectivity windows.

B1.3B - Starvation Prevention via Priority Aging

In strict priority queues, continuous arrival of high-priority telemetry could indefinitely starve low-priority daily summaries. Production mobile schedulers prevent starvation by adding an age-weighted boost (effective_priority = base_priority + alpha * waiting_hours) so long-deferred background reports eventually graduate into an eligible Wi-Fi/charging maintenance window.

B1.4 - Measurement and Interpretation

The program records enqueued, deferred, cancelled and successful work items, along with peak concurrent workers. The simulation uses one worker, so peak concurrency is one. With good connectivity and battery, both strategies may complete the same number of items; their main difference is execution order. Under restrictive conditions, the priority strategy should place critical items earlier, but it cannot bypass permission, encryption or authentication requirements.

TASK B2 - Adversarial Workloads

Two workloads can contain the same number of records but behave differently because their priorities, sizes and constraints differ.

Workload	Five-item design	Prediction
A - Easy	Five small records, valid permission, stable network, battery 80%, no Doze	Few deferrals and most records should be delivered successfully.
B - Difficult	Five records including large low-priority items, unstable/offline network, battery 15%, Doze enabled	More deferrals; only eligible critical work may proceed.

A suitable experiment is to run the same scheduler twice with five items in each workload and the same threshold values. Record processed items, deferrals and successful deliveries. The difficult workload has more constraint conflicts and therefore requires additional checks and later retries. Input size alone does not represent computational effort; the structure of the input and the frequency of constraint failures are also important.

B2.1 - Retry Storms, Thundering Herd Mitigation, and Token-Bucket Rate Limiting

When Workload B experiences prolonged offline or Doze periods, deferred tasks accumulate in the local queue. If connectivity is restored simultaneously across thousands of field devices:

- **Exponential Backoff with Decorrelated Jitter:** Retrying all failed records immediately upon connectivity restoration triggers a 'thundering herd' retry storm that saturates the cellular radio and backend API servers. Production schedulers pair truncated exponential

backoff (delay = min(max_delay, base * 2^attempt) + random_jitter) with a token-bucket rate limiter to smooth bursty post-Doze uploads.

- **Bounded Queue Compaction Under Sustained Adversity:** To prevent unbounded queue growth during multi-day network outages, the scheduler applies a bounded ring-buffer or priority-eviction policy that summarizes low-priority routine telemetry while preserving all high-priority clinical records.
- **Constraint-Failure Telemetry and Amortized Energy Cost:** Tracking constraint-check overhead separately from payload transfer time reveals that adversarial workloads incur higher CPU wake-up counts per delivered kilobyte due to repeated precondition evaluations and failed handshake attempts.

B2.2 - Adversarial Workload Telemetry Comparison

Metric	**Workload A (Easy)**	**Workload B (Adversarial)**	**Engineering Implication**
Delivered Items (Pass 1)	5 / 5 (100%)	1 / 5 (20% - critical only)	Deferral preserves battery under constraint stress
Deferred Items	0	4 (large/non-urgent)	Requires durable encrypted local storage
Wake-ups per KB Sent	Low (single burst)	High (repeated retries)	Must coalesce retries into Doze maintenance windows

Tail Latency (P99)	Low (< 500 ms)	High (hours until Wi-Fi)	Critical alerts must bypass routine batch queues

TASK B3 - Exact vs Approximate UI Optimisation

B3.1 - Example Cost Model

For a small simulation, define cost = 3 x frame drops + estimated energy units. A configuration is represented by quality level and cache size. The exact method evaluates every configuration and selects the minimum-cost feasible option. The approximate method starts with medium quality and a small cache, then adjusts quality and cache based on observed frame time and battery.

B3.2 - Python Exhaustive Search and Greedy Example

```
from itertools import product

def exact_search(frame_drop, energy, max_frame=16):
    best = None
    evaluated = 0
    for quality, cache in product([1, 2, 3], [1, 2, 3]):
        evaluated += 1
        frame = frame_drop[(quality, cache)]
        power = energy[(quality, cache)]
        if frame <= max_frame:
            cost = 3 * max(0, frame - 10) + power
            if best is None or cost < best[0]:
                best = (cost, quality, cache)
    return best, evaluated

def greedy_choice(frame_drop, energy, max_frame=16):
    quality, cache = 2, 2
```

```
    # One-step local adjustment based on current frame time.
    frame = frame_drop[(quality, cache)]
    if frame > max_frame:
        quality = 1
        cache = 1
    elif frame < 12:
        quality = min(3, quality + 1)
    cost = 3 * max(0, frame_drop[(quality, cache)] - 10) + energy[(quality, cache)]
    return (cost, quality, cache)


frame_drop = {
    (1,1): 11, (1,2): 10, (1,3): 10,
    (2,1): 15, (2,2): 14, (2,3): 13,
    (3,1): 20, (3,2): 18, (3,3): 17
}
energy = {
    (1,1): 2, (1,2): 3, (1,3): 4,
    (2,1): 4, (2,2): 5, (2,3): 6,
    (3,1): 7, (3,2): 8, (3,3): 9
}


print("Exact:", exact_search(frame_drop, energy))
print("Greedy:", greedy_choice(frame_drop, energy))
```

B3.3 - Three Test Instances and Counter-example

Instance	Exact method	Approximate	Interpretation

		method	
1 - Low load	Evaluates all 9 configurations and chooses minimum feasible cost	May choose the same low-quality, low-cache setting	Both can match
2 - Medium load	Finds the lowest-cost configuration under 16 ms	May settle on a nearby configuration	May differ slightly
3 - Heavy load	Finds the best feasible configuration if one exists	May reduce quality/cache quickly based on local measurements	Can be feasible but not optimal

A counter-example occurs when the greedy method reduces both quality and cache immediately after one slow measurement. A slightly larger cache with medium quality may have a lower total cost over the full feed, but the local decision does not examine that combination. The exact method can find it because it enumerates all small configurations.

B3.3A - Numerical Cost Evaluation Matrix and Time-Complexity Scaling

Evaluating the exact cost formula cost = 3 * max(0, frame - 10) + energy across the 3x3 parameter grid in B3.2 yields:

- **Numerical Comparison of Exact vs. Greedy Outputs:** Exact Search evaluates all 9 configurations: (1,2) achieves frame=10 ms and energy=3, producing the global minimum cost = 3*(0) + 3 = 3. Meanwhile, greedy_choice starts at (2,2) where frame=14 ms lies

between 12 ms and 16 ms, so it retains (2,2) with cost = 3*(14 - 10) + 5 = 17—a feasible frame time (<16 ms) that consumes 14 more cost units than the exact optimum.

- **Asymptotic Complexity Trade-Off:** For K tunable UI parameters each with M discrete levels, exhaustive search requires O(M^K) evaluations per adaptation window, whereas one-step greedy hill-climbing requires O(1) evaluations, making greedy adaptation mandatory for real-time 60/120 Hz rendering loops.
- **Hybrid Offline-Exact / Online-Greedy Tuning:** In production systems, engineers combine offline exact profiling across reference device tiers (generating baseline lookup tables) with lightweight online greedy adjustments to handle transient thermal or memory spikes.

B3.3B - Complete 3x3 Configuration Space Cost Breakdown

(Quality, Cache)	**Frame Time (ms)**	**Energy Units**	**Feasible (<= 16 ms)?**	**Total Cost = 3*max(0, frame-10) + energy**
(1, 1)	11 ms	2	Yes	3*(1) + 2 = 5
(1, 2)	10 ms	3	Yes	3*(0) + 3 = 3 (Exact Global Optimum)
(1, 3)	10 ms	4	Yes	3*(0) + 4 = 4
(2, 1)	15 ms	4	Yes	3*(5) + 4 = 19
(2, 2)	14 ms	5	Yes	3*(4) + 5 = 17 (Greedy Choice)
(2, 3)	13 ms	6	Yes	3*(3) + 6 = 15

(3, 1) to (3, 3)	17–20 ms	7–9	No (Exceeds VSYNC)	Infeasible (Dropped Frames)

B3.4 - Scalability and Engineering Trade-off

The exact method evaluates every combination, so its operation count grows with the number of quality levels, cache sizes and prefetch settings. The approximate method evaluates only a small number of candidate changes and is therefore more practical for runtime adaptation. It sacrifices a guarantee of global optimality. A real application may accept this trade-off because maintaining smooth scrolling and low battery use is more important than spending additional time searching for a perfect configuration.

TASK B4 - Multimedia Framework Selection

B4.1 - Practical Heuristic

A practical strategy selects video resolution, bitrate and upload timing from the device battery and network quality. A low battery or weak network selects a lower resolution and bitrate and defers upload when possible. A strong network and healthy battery permit higher quality. Video should be encoded using platform-supported multimedia APIs; Android uses MediaCodec and camera/media APIs, while Apple platforms provide AVFoundation and Core Media/Core Video components.

Sample device	**Battery**	**Network**	**Selected resolution**	**Bitrate**	**Upload decision**
Device A	85%	Strong Wi-Fi	1080p	4 Mbps	Upload now
Device B	45%	Moderate 4G	720p	2 Mbps	Upload when stable
Device C	15%	Weak mobile data	480p	0.8 Mbps	Defer non-urgent upload

B4.2 - Challenge the Strategy

A possible failure case is a device with a low battery but access to a very strong Wi-Fi connection and a short urgent clip. A simple rule may always select low resolution and defer upload, even though a short high-quality upload could finish quickly with acceptable energy use. This shows that fixed thresholds may be suboptimal. The strategy should consider clip duration, urgency, network throughput and estimated energy together.

B4.2A - Hardware Codec Acceleration (MediaCodec / VideoToolbox) and Adaptive Bitrate Streaming

Choosing between hardware-accelerated encoding and software codecs directly determines thermal throttling and battery drain during mobile multimedia capture:

- **Hardware-Accelerated Zero-Copy Video Pipelines:** On Android, configuring MediaCodec with MediaFormat.MIMETYPE_VIDEO_AVC (H.264) or MIMETYPE_VIDEO_HEVC (H.265) backed by an input Surface delegates YUV-to-bitstream compression directly to dedicated SoC DSP/ASIC video blocks via zero-copy GraphicBuffer handles. On iOS, AVAssetWriter and VideoToolbox (VTCompressionSession) perform hardware-accelerated HEVC encoding, cutting CPU utilization by over 70% compared to software libx264 encoding.
- **Segmented Chunked Upload and Energy-Per-Bit Utility Scoring:** To overcome the rigid static-threshold failure noted in B4.2, modern multimedia pipelines combine multi-dimensional utility scoring (E_tx = (Bitrate * Duration) / Throughput * P_radio) with segmented HLS/DASH chunking, uploading small 2-second encrypted segments incrementally so partial progress is never lost if cellular connectivity drops.
- **Media Metadata Scrubbing and Segment Encryption:** Before queuing video chunks for network transfer, stripping sensitive EXIF/GPS metadata containers (unless explicitly required by clinical/field workflow policy) and encrypting media segments with AES-GCM protects patient and user privacy.

B4.2B - Cross-Platform Multimedia Pipeline Comparison (Android vs. iOS)

Pipeli	Android Implementation	iOS Implementation	Optimiza

ne Stage			tion Impact
Camera Capture	Camera2 / CameraX ImageAnalysis & VideoCapture	AVCaptureSession / AVCaptureVideoDataOutput	Hardware sensor binning & zero-copy buffers
Hardware Encoding	MediaCodec (Surface input, H.264/HEVC)	VideoToolbox (VTCompressionSession)	Offloads CPU; prevents thermal throttling
Adaptive Bitrate	Dynamic MediaCodec.PARAMETER_KEY_VIDEO_BITRATE	Dynamic kVTCompressionPropertyKey_AverageBitRate	Adjusts mid-stream without keyframe reset
Chunked Transport	WorkManager + OkHttp HTTP/2 streaming	Background URLSession uploadTask	Survives app backgrounding & network handoffs

B4.3 - Production Engineering Decision Note

- Exhaustive search over every resolution, bitrate and upload-time combination is unattractive when thousands of devices need quick decisions.
- The heuristic provides a feasible choice according to its rules, but it does not guarantee a globally minimum energy configuration.
- It reduces decision time by checking a small number of device and network conditions instead of enumerating every possible combination.
- Exact search may be useful offline for small parameter spaces. Cloud-assisted analysis may help learn improved policies, but private video and device information must be protected.
- Before deployment, monitor encoding time, upload completion rate, bitrate, frame drops, battery change, data usage, failures and user-visible delay.

CONCLUSION

This assignment demonstrates that mobile application engineering requires more than making an application run. Background tasks must be scheduled according to battery, connectivity, permissions and security constraints. Synchronisation must be incremental and must preserve records safely when the device is offline. Sensor collection, data synchronisation and notification delivery share a common background-work pattern based on state, decisions, feasibility checks, execution and cancellation.

The comparison of local, cloud and hybrid processing shows that each approach has different advantages and limitations. Exact optimisation can find the best configuration for a small search space, while heuristic methods provide faster practical decisions for changing mobile conditions. Instrumentation and testing are necessary to confirm whether the design performs well. A reliable mobile application should balance responsiveness, energy use, security, usability and scalability.